THE QUANT CLUB · IIT (BHU) VARANASI

HIDDEN HAND

An Algorithmic Strategy Event

Different hands. Same rules. Only the sharpest minds win.

Dates	5 to 8 October 2026
Format	Teams of 3, one mentor per 3 to 4 teams
Contents	Event plan, game format and the full problem statement
Version	5 October 2026

Analyze. Develop. Outplay.

Contents

1 Summary **2**

2 Game format **2**

2.1 The deal . . . 2
2.2 Why this design . . . 3
2.3 Balance, measured . . . 3
2.4 Scoring: why a square root . . . 3

3 Problem statement **4**

3.1 The game . . . 4
3.2 A round . . . 4
3.2.1 Moves . . . 5
3.2.2 Trading . . . 5
3.3 A real deal . . . 5
3.4 Your bot . . . 5
3.5 House bots . . . 6
3.6 The arena . . . 6
3.7 What you submit . . . 7
3.8 Evaluation . . . 7
3.9 Rules . . . 7
3.10 Quick start . . . 8
3.11 FAQ . . . 8
3.11.1 Glossary . . . 8

4 The arena website **9**

5 Timeline and mentors **9**

6 Operations **10**

6.1 Setup on the laptop . . . 11
6.2 Last day . . . 11

7 Open items **11**

1 Summary

Hidden Hand is a four-day bot-trading event for first-year students, held 5 to 8 October 2026. Teams of three write a Python bot that trades a card-based stock against other teams' bots. It is our own game, built on the poster's cards theme, and not a copy of QuantStorm's *Divided Oracle*.

Who	First-year students, teams of 3, one mentor for every 3 to 4 teams
What they build	One Python file: a `Bot` class with three methods (`move`, `quote`, `respond`)
How it is judged	Every bot plays every other bot over hundreds of deals. Ranked on a luck-damped score: each deal pair's PnL on a square-root scale, averaged
Where they submit	Our own website, the *arena*. Uploads are checked at once, fights run automatically, results and replays are public, code stays private
Final result	A separate run on hidden deals after the deadline, so tuning to the live board does not help
What it teaches	Probability without replacement (card counting), reading information from prices, the value of information, risk limits

Ready: the game engine, starter bot, three reference bots, a local testing tool, the arena website, the full problem statement (Section 3) and a hosting plan that needs no AWS. All of it is tested end to end and lives in the `hiddenhand/` folder.

Not ready: links, times, prizes and contacts, listed in Section 7.

2 Game format

Two bots trade one stock whose value is the sum of 15 cards: both players' hidden hands plus a five-card board. Each bot knows only its own hand, so the game is about working out the opponent's hand from the way they trade.

2.1 The deal

One shuffled 52-card deck; each card is worth its rank, A $= 1$ up to K $= 13$. Each bot gets five private cards, and five board cards are dealt face down. The stock settles at the sum of all 15 cards, which is 105 on average. A deal has five rounds, and each round runs in this order:

1. A board card is turned face up (rounds 2 to 5; the fifth stays hidden to the end).
2. Both bots secretly choose a move: nothing, **PEEK** (pay 3 to see one random opponent card), or **SWAP** (discard a card face up and draw a new one; once per deal).
3. Bot X names a price. Bot Y may buy up to 2 lots at price $+2$, sell up to 2 lots at price -2, or pass.
4. The roles reverse: Y names a price and X trades.

Positions are capped at 10 lots either way. At the end every card is turned over and each lot pays the final sum. Trading is zero-sum; peek costs go to the house.

Fairness. Every deal is played twice with the hands swapped, so the luck of the deal cancels out. Two copies of the same bot always score exactly zero.

2.2 Why this design

Cards, not coins.
It matches the poster and is clearly different from Divided Oracle's ±1 coins. Cards are dealt without replacement, so card counting is a real skill: if you hold three kings, the opponent probably holds none.

Every move has a consequence.
A swap changes the value of the stock, so a bot that is long has a reason to throw away a low card. The discard is public, so the opponent can work out what happened.

Prices leak information.
A bot that quotes its honest estimate gives away its hand. Reading that leak, and hiding your own, is where good bots win.

Information has a price.
A peek costs 3 and the first one is worth about 4 before costs, less after that. Deciding when it pays is a real question.

2.3 Balance, measured

Each matchup was run as $3 \times 1{,}000$ deals; the figure is PnL per deal for the first-named bot.

Matchup	PnL per deal	What it shows
`starter_bot` vs `naive`	0.0	The starter is the baseline; teams must improve it
`reader` vs `naive`	+14 to +19	Reading the opponent's price is worth a lot
`sharp` vs `reader`	+15 to +16	Using the swap well is worth as much again
`sharp` vs `naive`	+30 to +34	Both skills together

The rule values were tuned for this balance. Two lots per trade with a limit of 10 spreads trading across all five rounds, and one swap per deal keeps swapping from drowning out everything else. A single deal can swing by 60 or more, so teams must test over about 1,000 deals.

2.4 Scoring: why a square root

Raw PnL lets one huge deal outweigh many ordinary ones, so the ranking uses a score instead. Each deal is played twice with the hands swapped; a bot's PnL over that pair is p, and the score is the average of $\text{sign}(p)\sqrt{|p|}$ over all deal pairs. The card luck cancels inside the pair first, and the square root then damps whatever luck is left. The square root has to be applied per deal pair: applied to a total it changes nothing, since the order of the totals stays the same.

Measured on 12 independent sets of 500 deal pairs each, using the same fight each time (signal-to-noise is the mean over the spread across the 12 sets; higher means luck matters less):

Fight	Raw PnL	Square root per pair	Square root per deal
`sharp` vs `reader` (clearly different)	13.2	**16.6**	12.9
`sharp` vs a near-copy of itself	0.7	**9.9**	0.2

With raw PnL two near-identical bots are a coin flip; with the score the better one wins reliably. Ties are broken on PnL per deal.

3 Problem statement

This section is the problem statement as participants read it. The same text ships as `README.md` in the kit.

Hidden Hand: Freshers' Algorithmic Strategy Event 2026

The Quant Club, IIT (BHU) Varanasi · 5 to 8 October 2026 · Teams of 3

You will write a Python bot that plays a card-trading game against other teams' bots. Every bot holds a hidden hand of cards, and they trade a stock whose value is set by everyone's cards. No finance background is needed. If you can write a Python class and you know what an average is, you can start today.

Build	One Python file: a `Bot` class with three methods
Play	Two bots, zero-sum, 5 rounds per deal, hundreds of deals per fight
Win	Highest score: your PnL (profit and loss) deal by deal, on a square-root scale so luck cannot decide it
Allowed	Python 3.10+, standard library only, AI tools
Submit	Upload to the arena (link on the WhatsApp community). Results appear within minutes

3.1 The game

The house shuffles one ordinary 52-card deck. Each card is worth its rank: A = 1, 2 to 10 at face value, J = 11, Q = 12, K = 13.

- **You** get 5 cards. Only you see them.
- **Your opponent** gets 5 cards. Only they see them.
- **The board** is 5 more cards, face down. One is turned face up at the start of each of rounds 2, 3, 4 and 5. The last one stays hidden until the end.

The stock you trade is worth **the sum of all 15 cards**: your hand, their hand and the whole board, which averages 105. At the end every card is turned over and every trade is paid out. If you bought below the final sum or sold above it, you made money, and your opponent lost exactly that amount.

3.2 A round

Every deal has five rounds. In each round a board card is turned (rounds 2 to 5), both bots choose a move at the same time, one bot quotes a price and the other trades on it, and then the roles swap.

Moves

Move	Cost	What it does
nothing	0	–
`"PEEK"`	3	You privately see one random card from the opponent's hand. They learn that you peeked, but not which card.
`("SWAP", i)`	0	Throw away card `i` of your hand face up and draw a new card that only you see. Once per deal.

A swap changes the stock's value. If you hold 4 lots and swap a 2 for a Q, everything you hold is worth 10 more per lot. Your opponent sees the 2 go but not what replaced it.

Trading

The *maker* names a single price. The *taker* may buy up to 2 lots at price $+2$, sell up to 2 lots at price -2, or pass. Every round each bot quotes once and trades once. Nobody may hold more than 10 lots long or short.

The maker's price is a clue. A maker usually quotes what it thinks the stock is worth, and that depends on its hand. The taker's choice to buy or sell is a clue too. Reading these clues is how you learn the hidden hand.

Fairness. Every deal is played twice with the hands swapped. Two copies of the same bot always score exactly zero.

3.3 A real deal

House bot `sharp` against house bot `reader`. Reproduce it with `python backtester.py sharp reader --deals 1 --show`.

`sharp` holds	Q♢ 4♣ 5♢ 10♠ 6♡	(sum 37)
`reader` holds	7♠ 4♡ 3♣ 7♣ 9♡	(sum 30)
Board, face down	A♣ 10♣ J♡ J♣ 8♣	

Round 1. `sharp` reckons $37 + 10 \times 6.96 = 106.6$ (ten unseen cards at the average unseen rank) and quotes that. `reader` thinks the stock is cheaper and sells 2 at 104.57. `reader` then quotes 101.04, and `sharp` buys 2 at 103.04.

Round 2. A♣ turns. `sharp` is long, so it swaps its 4♣ and draws a 7♢: +3 on everything it holds. It keeps buying from `reader`.

Rounds 3–5. Prices close in and trading stops. `sharp` finishes long 10.

Settlement. $40 + 30 + 41 = 111$. `sharp` bought at about 103 and makes +82. The same deal is then played again with the hands swapped.

3.4 Your bot

```python
class Bot:
    def move(self, obs):            # once per round, before trading
        return None                 # or "PEEK", or ("SWAP", index)
```

```
def quote(self, obs):           # you are the maker
    return 104.5                # they buy at 106.5 or sell at 102.5

def respond(self, obs, price):  # they quoted `price`
    return 2                    # +n buy n, -n sell n, 0 pass
```

A fresh `Bot()` is built for every deal. `obs` holds everything you are allowed to know:

Field	Meaning
`round`	1 to 5
`hand`	your 5 cards now, as ranks 1 to 13
`board`	board cards turned so far
`opp_known`	opponent cards you peeked that they still hold
`unseen`	`unseen[r-1]` = copies of rank `r` you have not seen anywhere (card counting, done for you)
`position`, `cash`, `spent`	lots held (+ long, − short), money from trades, paid for moves
`tape`	every quote so far: `(round, "me"/"opp", price, qty)`
`discards`	every swapped-out card: `(round, "me"/"opp", rank)`
`peeks`	every peek: `(round, "me"/"opp")`

A crash, an unusable return value or a call over 50 ms counts as doing nothing for that action. If your `quote()` fails, the house quotes 105 for you.

3.5 House bots

Three reference bots ship with the kit and play in the arena. You may copy anything from them.

Bot	What it does
`naive`	Counts cards: my cards + unseen cards × average unseen rank. Ignores the opponent and never moves. **The starter bot is `naive`.**
`reader`	`naive`, plus it estimates the opponent's hand from their last price
`sharp`	`reader`, plus it swaps a low card when long (a high one when short) and sizes trades by edge. Never peeks

They are far from the best possible bot. Things they never try: peeking, learning from the opponent's trades, expecting the opponent to swap, and quoting a price that hides their own hand.

Test properly. One deal can swing by 60 or more. Run 1,000 deals (about two seconds) on two or three seeds before you believe a change helped.

3.6 The arena

You submit and see results on the arena; its link is posted on the WhatsApp community.

1. Log in with the team key the organisers give you.
2. Upload your `.py` file. It is checked at once; a failed check shows why and does not count against your upload limit.

3. Your newest working file then plays every other team's newest file and the house bots on hidden deals. The live standings rank the score (see Evaluation), and you can watch the first deal of any fight, round by round.

Code is never shown to anyone; results and replays are public. You may upload once every 10 minutes, and your newest working file is your entry.

The final. After the deadline every entry plays every other on a different hidden set of deals. Only the final decides the result.

3.7 What you submit

One `.py` file per team, made from `starter_bot.py`, which:

- starts with these three lines, filled in:

```
# Team Name: Your Team Name
# Members: Name 1 (Roll No.), Name 2 (Roll No.), Name 3 (Roll No.)
# Mentor: Your Mentor's Name
```

- defines `Bot` with `move`, `quote` and `respond`;
- imports only `math`, `random`, `statistics`, `collections`, `heapq`, `bisect`, `itertools`, `functools`, `typing`;
- passes `python backtester.py --validate your_bot.py`;
- keeps nothing from one deal to the next.

3.8 Evaluation

You are ranked on a **score** built deal by deal, so that one lucky deal cannot decide anything.

1. Every deal is played twice with the hands swapped. Add your PnL from the two plays and call it p. The luck of the cards mostly cancels here.
2. Take the signed square root of p: a deal pair won by 100 counts $+10$, one won by 25 counts $+5$, one lost by 16 counts -4.
3. Your score is the average of these over every deal pair of every final fight:

$$\text{score} \;=\; \frac{1}{N}\sum_{i=1}^{N} \operatorname{sign}(p_i)\,\sqrt{|p_i|}$$

A single huge win, say a lucky last card while you are long 10, counts for far less than its raw PnL, while winning deal after deal counts in full. Tested over 12 sets of 500 deals, this separates a better bot from a worse one far more reliably than raw PnL. `backtester.py` prints your score, so you can tune for exactly what is ranked.

Ties are broken on PnL per deal. Every bot plays every other bot and the three house bots the same number of deals. House bots do not take a place.

3.9 Rules

1. Your bot is your team's own work. Discussing ideas is fine; sharing code or another team's tuned numbers is not. The starter and house bots are free to copy.

2. AI tools are allowed. Any member must be able to explain any part of the file.
3. Mentors guide; they do not build. A bot written or tuned by a mentor is disqualified.
4. Play the game, not the system. Reading files, using the network or keeping state between deals gets the file rejected. Report loopholes; do not use them.
5. One team, one entry. A member may be in only one team.
6. The organisers' decision is final. Ask on the group before building on a guess.

3.10 Quick start

Download the kit from the arena (*Kit*, top bar), unzip it, then:

```
cd hiddenhand                       # Python 3.10+, nothing to install
python backtester.py --show         # sharp vs reader, one deal printed
cp starter_bot.py my_bot.py         # your entry starts here
python backtester.py my_bot.py naive --deals 1000
python backtester.py --validate my_bot.py
```

3.11 FAQ

I have never traded. Is that a problem?
No. It is a probability game more than a finance one.

Can I use numpy or pandas?
No, standard library only. The game is small enough not to need them.

How fast must my bot be?
A millisecond or two per call. A call over 50 ms counts as doing nothing.

Why did my rank change when I did not upload?
Other teams uploaded, and your bot now plays their new bots.

Where do I ask?
Your mentor first, then the WhatsApp community. Rule answers given there apply to everyone.

Glossary

PnL	Profit and loss: how much you made (positive) or lost (negative)
Long / short	Long: you bought, so you gain if the sum ends higher. Short: you sold
Lot	One unit of the stock; each lot pays the final sum
Maker / taker	The maker names the price; the taker decides whether to trade
Spread	The gap between the buy price (price $+2$) and the sell price (price -2)
Card counting	Tracking which cards are gone, so you know what is left

4 The arena website

The arena is our own submission and results site, run like a closed test set: teams never see each other's code or the hidden deals, only the scores and the replays. It is one Python file (`arena.py`, standard library only) with a SQLite database.

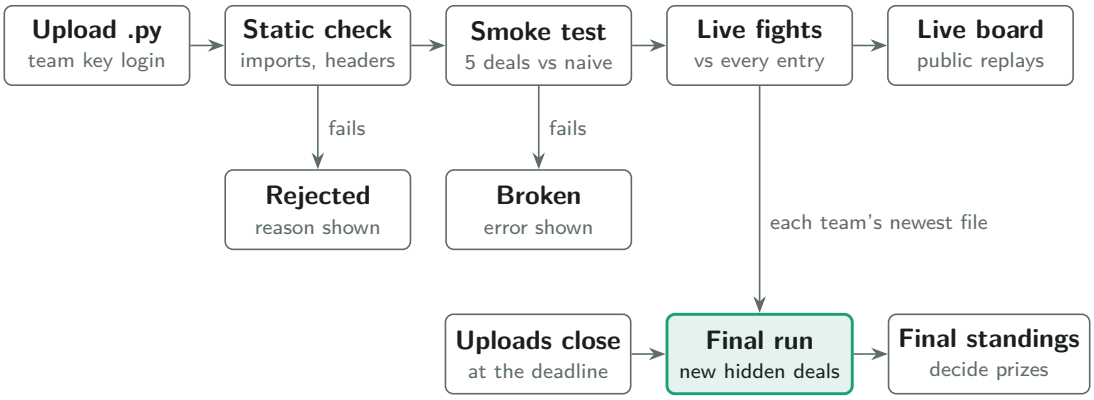


Figure 1. The submission pipeline. Only the final run on hidden deals decides the ranking.

A failed upload shows its reason at once and does not count against the 10-minute upload limit. Live fights use one hidden seed and the final uses a different one, so nobody can tune to the board.

Page	Who sees it	What it shows
Standings (home)	Everyone	Live board ranked by score, with PnL per deal, fights won and version; the final table on top once it is run; latest fights
Team page	Everyone	That team's head-to-head result against every other entry
Fight replay	Everyone	The first deal of a fight both ways round: hands, board, moves, prices, trades, final PnL
Submit	That team only	Upload form, every upload with its status and reason, any timeouts or crashes in its fights
Kit	Everyone	Zip of the starter bot, house bots, engine, backtester and problem statement

Safety. Every submission runs in its own locked-down process: no files, no network, no imports outside the allowed list, 50 ms per call, a memory cap, and a full wipe between deals. The engine's cards live in a different process, so a bot cannot read them.

House bots. `naive`, `reader` and `sharp` appear on the board, greyed out, as benchmarks. They do not take a rank.

5 Timeline and mentors

The arena opens at kickoff on Monday 5 October and closes on Thursday 8 October, when the final is run and the results are announced.

Day	What happens	Milestone for each team	Organisers do
Mon 5 Oct	Kickoff: the game, the kit, meet your mentor. Arena opens	Run `backtester.py --show` and explain one deal end to end	Send team keys; post the arena link and kit
Tue 6 Oct	Mentor check-in 1	A bot that beats `naive` and is on the live board	Watch for broken uploads; answer rule questions
Wed 7 Oct	Mentor check-in 2	Beats `reader`; knows where it loses PnL	Announce the exact deadline again
Thu 8 Oct	Uploads close at **[TBD: time]**. Final run; results at **[TBD: time, venue]**	Newest upload is the one they want	Run the final, back up the data, present results with replays

Mentors look after 3 to 4 teams each and hold a 30-minute check-in with every team on Tuesday and Wednesday.

Mentors may	Mentors may not
Explain the rules, probability, card counting and the engine	Write or edit a team's bot
Ask "why?" until the team finds its own bug	Suggest specific numbers, weights or thresholds
Point to the problem statement, house bots and replays	Pass one team's ideas or numbers to another
Show how to run a fair test (1,000 deals, several seeds)	Run a team's tuning for them

Before kickoff every mentor should read the problem statement, play through `python backtester.py --show`, and be able to explain three things without notes: why the unseen cards are worth the average unseen rank, why a maker's price leaks its hand, and why 100 deals prove nothing. Suspected copying, or a team that cannot explain its own code, goes to the coordinators, not to the team.

6 Operations

The arena needs no AWS: one machine that stays on for four days and a public link. The recommended setup is a club laptop with a free Cloudflare tunnel; an Oracle Cloud free VM is the sturdier alternative.

Option	Cost	Setup	Watch out for
Club laptop + Cloudflare tunnel (recommended)	Free	5 min	Must stay on, awake and online; the link changes on restart
Oracle Cloud Always Free VM or a college server, with Docker	Free	30 min	Open port 80 in the VM firewall
Railway or Fly.io from the Dockerfile	A few $	15 min	Attach a volume at `/data`, or results are lost on redeploy

Do not use Render's free tier: it sleeps when idle and wipes its disk.

6.1 Setup on the laptop

```
winget install Cloudflare.cloudflared     # once
cd hiddenhand
python arena.py add-team "Team Name"      # once per team; prints its key
set DEADLINE=2026-10-08T18:00             # uploads close by themselves
python arena.py --tunnel                  # prints the public https link
```

Send each team its key privately and post the link on the group. Set the laptop never to sleep, and allow Python through the firewall if Windows asks. `python arena.py teams` lists every team and key again.

Settings (environment variables, all optional): `DEADLINE` closes uploads; `DEALS` = 200 deals each way per live fight; `FINAL_DEALS` = 500; `COOLDOWN` = 600 seconds between uploads; `WORKERS` = fights run at once (CPU cores minus one).

6.2 Last day

1. At the deadline uploads close automatically.
2. Run `python arena.py final` on the arena machine. With 30 teams that is about 500 fights; with 3 workers allow 15 to 30 minutes.
3. The final table appears at the top of the site at once. Open the top fights' replays for the results presentation.
4. Copy the `arena_data` folder somewhere safe. It holds every upload and every result.

The full guide, including the Docker commands, is `DEPLOY.md` in the `hiddenhand/` folder.

7 Open items

The game, the problem statement and the site are final. These details must be filled in before the problem statement goes out; each is marked **[TBD: ...]** in this document and in the README.

- ☐ Choose the hosting option, start the arena and put its link in the problem statement
- ☐ Exact upload deadline on Thu 8 Oct (also set as `DEADLINE`)
- ☐ Results announcement time and venue
- ☐ Registration link and registration deadline
- ☐ Prizes
- ☐ Coordinator names and phone numbers
- ☐ Mentor list and team-to-mentor assignment
- ☐ One team key per registered team, sent privately
- ☐ Retire the old Divided Oracle problem statement and the `freshers-ps/` folder